AI Test Plan Generator - Independent Project Audit

Date: 2026-06-18

Source of truth: codebase under `/Users/tahaelyounsi/Desktop/repos/ai-testplan-generator`

Reference vision: `Cahier_des_charges_SIGMAXIS_ENSAM_P5.docx (1).md`

1. Executive Summary

This project is not just a README or mockup. It contains a real FastAPI backend, a real React/Vite frontend, domain models, document loaders, chunking, LLM-based requirement extraction, LLM-based test generation, a traceability graph, defect taxonomy/static checks, authentication scaffolding, API key support, background-job abstractions, observability, Docker/Compose/Helm files, and a usable UI shell.

But it is not a reliable product MVP yet. The strongest parts are the backend AI pipeline shape, typed models, defect engine, and test coverage around the in-memory/dev path. The weakest parts are persistence, production wiring, authentication/authorization, general knowledge management, contextual chat consistency, project membership bootstrapping, and frontend/backend contract drift.

The central product contradiction is this: the code presents itself as a traceable, multi-project AI test-plan platform, but most of the product-critical artefacts are still in process memory. Projects/users/API keys are SQLite-backed, and blobs can be persisted, but documents, chunks, extracted requirements, generated plans, project-plan indexes, job checkpoints, and most traceability state are not durably represented in the application database. Persistent vector/graph backends exist as adapters, but the API still depends on the in-memory object registry in `MemoryManager`.

The current state is best described as a strong technical prototype or internal demo MVP foundation, not a production MVP for industrial test-plan work.

2. What Was Actually Achieved

Area	Actual state
Backend architecture	Real FastAPI app factory with routers for auth, documents, plans, chat, traceability, projects, events, admin, quality, and health. See `src/ai_testplan_generator/api/app.py`.
Document ingestion	Real loaders for PDF, DOCX, XLSX/XLSM, Markdown, and text; chunking preserves sections, pages, and source ids. Evidence: `ingestion/loaders.py`, `ingestion/chunking.py`, `ingestion/pipeline.py`.
AI extraction/generation	Real LiteLLM gateway and typed structured-output calls for requirement extraction, plan architecture, test generation, review, planning, and copilot. Evidence: `llm/litellm_gateway.py`, `agents/*`, `prompts/library.py`.
Traceability model	Real graph links exist: document -> section -> chunk -> requirement -> test case, plus coverage matrix calculation. Evidence: `memory/manager.py:140-294`, `memory/cross_document.py`.
Test plan models	Reasonably rich `TestPlan`, `TestCase`, `TestStep`, `AcceptanceCriterion`, schedule, resource, and defect models. Evidence: `models/tests.py`, `models/planning.py`, `models/defects.py`.
Quality engine	Real static checks and defect aggregation exist. The static eval passed on the bundled synthetic benchmark. Evidence: `quality/static_checks.py`, `agents/defect_aggregator.py`, `evals`.

Area	Actual state
Frontend	Real React UI exists for projects, uploads, plans, interactive runs, chat, traceability, general KB, API keys, admin, and PDF/JSON export. Evidence: `frontend/src/features/*`.
Tests	`.venv/bin/python -m pytest -q` passed: 193 passed, 6 skipped. Frontend `npm test -- --run` passed: 11 tests. `npm run build` succeeded. Static eval passed with 100% defect recall on the bundled synthetic benchmark.

3. Biggest Gaps And Risks

3.1 Persistence Is Not Product-Grade

The app creates durable SQLite repositories for projects and users, but product artefacts are stored in memory:

- `api/app.py:105-108` initializes `app.state.jobs`, `app.state.plans`, `app.state.project_plans`, and `app.state.defects` as dictionaries.
- `api/deps.py:49-62` exposes these dictionaries as dependencies.
- `memory/manager.py:63-72` defines `_Artefacts` as the object registry for documents, sections, chunks, requirements, test cases, and plans.
- `memory/manager.py:140-294` writes all structural artefacts to this in-memory registry before also adding graph/vector entries.
- `memory/manager.py:345-358` API-facing reads for documents and requirements query `_store`, not SQLite/Qdrant/Neo4j.

Conclusion: a restart loses the application's working knowledge of documents, chunks, requirements, and generated plans unless the same process memory remains alive. Blob files may remain on disk/S3, but the app cannot reconstruct the product state from blobs alone.

There is also a specific document-storage bug: `_stream_to_blob` stores uploaded bytes under a blob key such as `projects/{project_id}/docs/{sha256}/{filename}` (`api/routers/documents.py:70-82`), but `_ingest_from_bytes` ignores the `blob_uri` parameter (`api/routers/documents.py:85-107`). The document loader then records `source_uri=path.resolve().as_uri()` for the temporary ingest file (`ingestion/loaders.py:53-69`). Download and delete later call `blob_store.presign_get(doc.source_uri)`, `blob_store.get_stream(doc.source_uri)`, and `blob_store.delete(doc.source_uri)` (`api/routers/documents.py:231-270`). That means the app saves the original upload under one key but tries to read/delete using a temporary `file://` URI. Upload/analysis can work while download/delete of original bytes is likely broken or at least inconsistent.

3.2 Production Job Queue Wiring Is Contradictory

The code supports both `FakeJobQueue` and ARQ/Redis, but the selection is coupled to semantic memory, not an explicit queue setting:

- `api/app.py:113-123` uses `FakeJobQueue` whenever `SEMANTIC_MEMORY_BACKEND=inmemory`.
- `docker-compose.yml:19` and `docker-compose.yml:53` set `SEMANTIC_MEMORY_BACKEND=inmemory` for both API and worker.
- `docker-compose.yml:41-65` starts a worker anyway.
- `jobs/worker.py:104` registers only `ingest_document`, `run_autonomous`, and `delete_project_artefacts`, not `run_autonomous_interactive`.

Conclusion: default Compose starts a worker that the API will not use. If you switch away from in-memory semantic storage so ARQ is used, the worker writes plan JSON to blob storage, but the API plan list/detail endpoints still rely on API-process dictionaries.

3.3 Auth/RBAC Is Partial And Inconsistent

There is real JWT/API-key code, but enforcement is uneven:

- `api/deps.py:69-115` resolves users from Bearer JWT or `X-Api-Key`.

- `api/security/rbac.py:26-35` defines useful project permissions.
- Protected endpoints exist for upload, delete, plan read/generate, and general KB upload.

But several sensitive routes are unprotected:

- Project create/list/get/update/archive/member add/list/remove have no auth dependencies in `api/routers/projects.py:99-190`.
- Document list/get/download have no RBAC dependency in `api/routers/documents.py:164-244`.
- Traceability endpoints have no auth dependency in `api/routers/traceability.py`.
- Chat, history, job status, SSE, and job checkpoint/resume are not protected in `api/routers/chat.py`, `api/routers/events.py`, and `api/routers/plans.py:206-240`.

Admin also does not work as a persisted concept:

- `domain/users.py:22-30` has no `is_admin` column.
- `domain/users.py:46-54` has an in-memory `is_admin` dataclass field.
- `_row_to_user` in `domain/users.py:205-214` never sets `is_admin`.
- `/auth/me` schema omits `is_admin` in `api/schemas/auth.py:47-52`.
- The frontend hides admin unless `user.is_admin` is truthy in `frontend/src/app/layout.tsx:55` and `frontend/src/features/admin/admin-page.tsx:15`.

Conclusion: security is not production-ready. Some RBAC exists, but many routes bypass it, and persisted admin users cannot be represented by the current schema.

3.4 New Projects Are Not Usable Under RBAC Without Manual Repair

The frontend creates projects with only name/description:

- `frontend/src/features/projects/api.ts` posts `{ name, description }`.
- Backend `create_project` accepts `owner_id` from request body and stores it, but does not use the authenticated user or add a project member (`api/routers/projects.py:99-108`).
- RBAC checks do not inspect `projects.owner_id`; they only check `project_members` (`api/security/rbac.py:60-64`).

Conclusion: a newly created project has no guaranteed owner/member row, so protected actions such as upload and plan generation can fail unless a member is manually added. The member endpoint itself is currently unprotected, which masks the broken ownership model rather than solving it.

3.5 General Knowledge Base UI Is Disconnected

General KB upload and list/delete use incompatible identities:

- General upload calls `/general/documents` in `frontend/src/features/knowledge/api.ts:25-43`.
- General list/delete calls `/projects/general/documents` and `/projects/general/documents/{docId}` in `frontend/src/features/knowledge/api.ts:16-23` and `46-48`.
- Backend general ingest creates documents with `project_id=None`, `scope="general"` in `knowledge/general.py:24-32` and `ingestion/loaders.py:53-69`.
- `/projects/{project_id}/documents` lists documents by `doc.project_id == project_id` via `memory/manager.py:345-346`.

Conclusion: uploaded general KB documents can be indexed for retrieval, but the frontend will not list or delete them because it queries project id `"general"` while the documents have `project_id=None`.

3.6 Chat Is Only Sometimes Contextual

The HTTP `/chat` path uses `CopilotAgent`, which retrieves project/general context:

- `api/routers/chat.py:43-55` calls `InteractivePipeline`.

- `agents/copilot.py:55-79` retrieves chunks/requirements and grounds the LLM prompt.

But the normal frontend message path uses WebSocket streaming:

- `frontend/src/features/chat/chat-page.tsx:172-202` sends slash commands through HTTP `/chat`.
- Non-slash messages use WebSocket after line 204.
- Backend WebSocket `api/routers/chat.py:92-127` sends only prior episodic history plus the current message to `brain.llm.stream`; it does not retrieve project context and does not know project id.
- `graphs/interactive.py:44-48` also confirms that mutating copilot actions are not applied.

Conclusion: the contextual chatbot requirement is partially implemented. Slash commands can be grounded. Normal chat is not project-grounded.

3.7 Planning Exists As Generated Data, Not As A Product Feature

The backend has a `PlannerAgent` and schedule model:

- `models/planning.py` defines `Resource`, `Milestone`, `TestSchedule`, and `ScheduledAssignment`.
- `agents/planner.py` asks an LLM for milestones and assignments.

But the pipeline calls the planner with `resources=[]`, and no route persists or displays schedules:

- `graphs/autonomous.py` and `pipelines/interactive_run.py` invoke `PlannerAgent.Input(plan=..., resources=[])`.
- There is no frontend Gantt, assignment, resource management, reminders, or execution tracking.

Conclusion: planning is an internal generated object, not a usable planning/follow-up feature.

4. Most Serious Contradictions

Claimed or implied capability	Actual evidence	Conclusion
Production-ready background processing	Compose starts a worker, but API uses `FakeJobQueue` when semantic memory is in-memory; worker lacks interactive task.	Queue system is prototype/dev by default, not production-consistent.
Persistent multi-project knowledge	Documents/requirements/plans are read from `MemoryManager._store`.	Knowledge exists only for the lifetime of the process unless rebuilt manually.
Document download/delete	Upload stores bytes by blob key, but metadata stores a temporary `file://` source URI and download/delete use that URI.	Original document retrieval is likely broken even before considering restart persistence.
General KB management	Upload uses `/general/documents`; list/delete use `/projects/general/documents`; general docs have `project_id=None`.	UI cannot manage uploaded general KB docs.
Role-based security	RBAC exists for selected routes, but many sensitive project/document/trace/chat/job routes are unprotected.	Security is a scaffold, not a coherent access model.
Admin area	UI expects `user.is_admin`, backend `/auth/me` does not return it, DB does not store it.	Admin is mostly reachable only through tests/stubs or custom out-of-band code.
Human-in-loop generation	Interactive task exists in `FakeJobQueue`; worker does not register it; paused state lives in memory.	Interactive mode is a dev/local feature, not production-ready.
Contextual chatbot	CopilotAgent retrieves context, but normal WebSocket chat bypasses it.	The most visible chat path is not actually contextual.
Revision loop	Orchestrator may route back to generator after findings, but `TestGeneratorAgent` receives only requirements/detail/user feedback, not reviewer findings in normal autonomous mode.	Automated revision is weak; it can regenerate without using the critique.

5. Feature Status Against The Specification

I used the cahier des charges as the intended product scope only. The key requirement clusters in `/Users/tahaelyounsi/Downloads/Cahier_des_charges_SIGMAXIS_ENSAM_P5.docx (1).md` are: automatic test-plan generation, general/project knowledge, source traceability, field-usable test instructions, summary/detailed outputs, chatbot validation, planning/follow-up, and technical/security evaluation (`lines 17-24`); document analysis, knowledge management, contextual chatbot, and test planning/follow-up in scope (`lines 30-35`); source-linked tests and summary/detailed levels (`lines 43-51`); user-selected detail and chatbot validation (`lines 53-57`); dates, milestones, resources, dashboards, Gantt, relances (`lines 59-63`); hardware, deployment timing, cloud security, role rights, confidentiality, and expected deliverables (`lines 65-85`).

Intended requirement	Actual code evidence	Status
Analyze uploaded technical documents	Real loaders/chunkers/extractors exist: `ingestion/loaders.py`, `ingestion/chunking.py`, `ingestion/pipeline.py`. Upload endpoint ingests small files synchronously and large files via queue.	Partial-real. Works in dev path, but metadata is not durable and original-file download/delete uses the wrong storage identity.
Generate test plans from uploaded documents	`plans.py:53-75` enqueues generation; `AutonomousPipeline` loads registered docs/requirements; `TestArchitectAgent` and `TestGeneratorAgent` call LLMs.	Real but fragile. Requires in-memory project docs/requirements.
Generate detailed test instructions	`TestCase` includes steps, criteria, equipment, teardown, duration, deliverables, dependencies, KPIs. `TestGeneratorAgent` fills these fields.	Real at model/prompt level. Quality depends on LLM output.
Summary vs detailed output	`DetailLevel` exists and is passed to prompts. Frontend toggles summary/full.	Partial. The frontend/backend response shapes do not fully align.
Trace every test to source documents	Requirement-to-chunk and test-to-requirement links exist. Test-to-source lineage is indirect through graph ancestors.	Partial-real. Coverage is real; source citation UX and persistence are weak.
General and project-specific knowledge	Namespaces exist: project chunks by project id and general chunks as `chunks:general`.	Partial. Retrieval can use both, but general KB management UI is broken.
Contextual chatbot	HTTP CopilotAgent retrieves context. WebSocket chat does not.	Partial/misleading.
Validation via chatbot	Copilot can propose pending actions, but `maybe_apply` is a no-op.	Mostly fake/incomplete.
Plan and follow test execution	Schedule model and planner agent exist. No execution tracking, Gantt, resources UI, assignments workflow, relances.	Mostly missing.
Dashboards	Project dashboard shows counts/coverage. Grafana dashboards exist for ops. No product execution dashboard.	Partial/UI-light.
Roles and rights	RBAC dependency exists for some routes.	Partial and unsafe.
Security/cloud confidentiality	JWT/API keys, optional blob encryption, CORS config, S3 storage exist.	Partial. Defaults and enforcement are not production-grade.
Hardware/deployment estimates	Docker/Compose/Helm exist. No feature estimates hardware requirements or client deployment duration.	Missing.
User guide/training	No real guide/training workflow found in code.	Missing.

6. Feature Classification

Feature	Complete	Partial	Fake/UI-only	Missing/Broken	Notes
Project CRUD		X			SQLite-backed, but unauthenticated and creator ownership broken.
Project members/RBAC		X			RBAC exists; member management is unauthenticated.
Document upload		X		X	Real ingest, but no durable document index, some read routes unprotected, and original-file download/delete likely uses the wrong blob key.
General KB upload		X		X	Ingest path exists; list/delete UI uses wrong project id.

Feature	Complete	Partial	Fake/UI-only	Missing/Broken	Notes
Requirement extraction		X			Real LLM extraction; failures per chunk are silently dropped.
Test plan generation		X			Real in dev path; production worker/API index mismatch.
Traceability coverage		X			Real graph coverage; persistence and source UX weak.
Defect engine	X				One of the strongest areas; static eval passes.
Interactive checkpoints		X			Good local prototype; not ARQ/persistent.
Chatbot		X			Grounded only on HTTP/slash path; stream path ungrounded.
Mutating copilot actions			X		Pending actions exist, apply step is no-op.
Planning/Gantt/follow-up		X	X		Schedule object exists; product workflow missing.
API keys		X			Real hashing/storage; UI text says Bearer but backend expects `X-Api-Key`.
Admin			X	X	Backend route guard exists, but persisted admin role absent and UI hidden.
Deployment		X			Docker/Helm present; queue/persistence configuration contradictory.

7. Technical And Product Critique

Good Decisions Worth Keeping

- The domain models are typed and fairly expressive. `TestPlan`, `TestCase`, `Requirement`, `TraceLink`, and `DefectReport` are useful foundations.
- The LLM provider boundary is clean. Agents depend on `LLMGateway`, not raw provider SDKs.
- The ingestion pipeline has a sensible shape: load -> chunk -> register -> extract -> deduplicate.
- The traceability graph concept is correct for this product; vector search alone would not satisfy auditability.
- The defect taxonomy/static checks are valuable and testable. This is the most product-specific "real IP" in the repo.
- The frontend already covers many intended workflows and is not merely a mockup.

Architectural Problems To Fix Before Scaling

1. Persist product artefacts in a real domain store.

Projects/users are not enough. You need durable tables or document-store records for documents, sections, chunks metadata, requirements, plans, test cases, schedules, trace links, defects, chat sessions, and run/checkpoint state.

2. Separate queue configuration from semantic memory configuration.

`SEMANTIC_MEMORY_BACKEND` should not decide whether jobs run in-process or through ARQ.

3. Rebuild auth/RBAC around authenticated ownership.

Project creation should use `current_user`, create an owner member row, and all project-scoped routes should require project membership.

4. Make general KB a first-class scope.

Do not pretend it is project id `"general"` in the frontend unless the backend also stores it that way. Prefer explicit `/general/documents` list/delete endpoints or a scope filter.

5. Make one chat path.

Either stream through CopilotAgent with retrieval or disable streaming until it is grounded. Two paths with different behavior will confuse users.

6. Decide whether planning is in scope now.

If yes, add resources, schedule persistence, assignments, statuses, Gantt, and follow-up. If no, hide schedule claims from product language.

7. Stop treating comments/README milestones as implementation truth.

Several files are organized around milestone labels, but the real runtime behavior lags behind those labels.

8. Practical Priority Roadmap

Phase 0 - Stabilize The Existing Prototype

1. Fix project ownership: require auth on project create; set `owner_id=current_user.id`; add owner membership immediately.
2. Protect all project/document/trace/chat/job routes or explicitly mark them public. Start with project CRUD, member management, document list/download, traceability, job status/checkpoint/resume, and chat history.
3. Add `is_admin` or a roles table to the persisted user schema, return it from `/auth/me`, and update tests to avoid admin stubs except where intentional.
4. Fix General KB list/delete contract.
5. Store the upload blob key on document metadata and make download/delete use that key instead of the temporary loader URI.
6. Fix frontend plan detail typing so full `TestPlan` does not masquerade as `TestPlanSummary`.
7. Fix API key docs/UI: backend uses `X-Api-Key`, while frontend says `Authorization: Bearer`.

Phase 1 - Make Data Durable

1. Add durable persistence for document metadata, sections/chunks metadata, extracted requirements, plans, test cases, coverage, defects, and schedules.
2. Rebuild `MemoryManager` so vector/graph stores are retrieval indexes, not the only place that can answer product queries.
3. Persist job/run records and interactive checkpoint state.
4. Add migration/versioning strategy for the domain DB instead of scattered schema creation across repositories.

Phase 2 - Make The AI Product Trustworthy

1. Make test generation fail loudly when extraction/generation produces empty outputs; silent chunk failures should be surfaced as warnings.
2. Feed reviewer findings into regeneration in autonomous mode.
3. Store citations/source excerpts on requirements/test cases in a way the UI can show without graph spelunking.
4. Add acceptance tests with a small real document fixture: upload -> extract -> generate -> inspect plan -> trace source -> restart -> data still exists.
5. Evaluate grounding quality, not just static defect recall.

Phase 3 - Decide Product Scope

1. If planning/follow-up is required: build resources, assignments, status tracking, Gantt, notifications/relances, and dashboard views.
2. If chatbot validation is required: implement confirmed mutations against persisted plans and audit every change.

3. If deployment estimation/hardware sizing is required: add an explicit estimator or remove the implied feature from scope.
4. If this is for industrial/security-sensitive use: harden CORS, secrets, token revocation, data isolation, encryption, audit access, and tenant boundaries.

9. Verification Performed

- `.venv/bin/python -m pytest -q`: 193 passed, 6 skipped, 7 warnings.
- `npm test -- --run` in `frontend`: 2 files passed, 11 tests.
- `npm run build` in `frontend`: succeeded; Vite warned about a large JS chunk.
- `PYTHONPATH=src .venv/bin/python -m evals.cli --static-only`: synthetic aerospace benchmark passed, defect recall 100.0% (6/6).

Environment note: the system/base Python lacked backend dependencies (`pytest_asyncio`, `structlog`), but the repo `.venv` had them and passed tests.

Worktree note: before the audit, `src/ai_testplan_generator/quality/static_checks.py` was already modified. Running the frontend build updated `frontend/tsconfig.tsbuildinfo`.

10. Evidence Appendix

Evidence	What it proves
`api/app.py:80-146`	App composes brain, repositories, event broker, in-memory plan/job/defect dictionaries, and queue selection.
`api/deps.py:49-62`	Plans, project plan index, jobs, and defects are pulled from `app.state` dictionaries.
`memory/manager.py:63-92`	Structural product artefacts live in `_Artefacts` in memory.
`memory/manager.py:140-294`	Documents, chunks, requirements, test cases, and plans are registered into `_store` and graph/vector stores.
`memory/manager.py:345-358`	API document/requirement reads use `_store`, not a durable domain DB.
`ingestion/pipeline.py:49-105`	Real ingest pipeline loads, chunks, registers, extracts, deduplicates, and registers requirements.
`ingestion/extraction.py:72-107`	Requirement extraction is LLM-structured and attaches source document/section/chunk IDs.
`api/routers/documents.py:70-107`, `api/routers/documents.py:231-270`, `ingestion/loaders.py:53-69`	Upload stores bytes under a blob key, but document metadata stores a temporary `file://` URI that download/delete later use as if it were the blob key.
`agents/test_generator.py:102-147`	Test generation retrieves source chunks and related context before structured LLM generation.
`agents/traceability.py:45-117`	Traceability agent validates coverage and computes coverage matrix from graph links.
`api/routers/plans.py:78-172`	Plan list/detail rely on in-memory `plans`/`project_plans`; only export has blob fallback.
`jobs/tasks/autonomous.py:61-79`	Generated plans are written to blob storage and also in-process indexes.
`jobs/worker.py:97-105`	ARQ worker does not register the interactive generation task.
`docker-compose.yml:19-22`, `docker-compose.yml:41-65`	Compose runs worker but config makes API choose FakeJobQueue.
`api/security/rbac.py:26-69`	RBAC roles/permissions exist but depend only on project member rows.
`api/routers/projects.py:99-190`	Project and member endpoints are not protected by auth/RBAC dependencies.
`domain/users.py:22-54`, `domain/users.py:205-214`	User table does not persist admin role; dataclass admin flag defaults false after DB load.
`api/schemas/auth.py:47-52`	`/auth/me` does not expose `is_admin`.
`knowledge/general.py:24-32`	General KB docs are ingested with `project_id=None`, `scope="general"`.
`frontend/src/features/knowledge/api.ts:16-48`	Frontend lists/deletes general docs through `/projects/general/documents`, which does not match backend storage.

Evidence	What it proves
`api/routers/chat.py:92-127`	WebSocket chat path streams from LLM using only chat history/current message, no project retrieval.
`agents/copilot.py:55-79`	HTTP CopilotAgent path does perform retrieval and grounding.
`graphs/interactive.py:44-48`	Copilot mutations are not actually applied.
`frontend/src/features/plans/plan-detail.tsx:22-83`, `api/schemas/plans.py:79-94`, `models/tests.py:85-113`	Frontend expects `n_test_cases` on both summary/full plan, but full backend `TestPlan` does not define it.
`tests/api/conftest.py:1-13`, `tests/api/conftest.py:91-110`	API tests bypass lifespan and override current user with an admin stub, masking auth/runtime issues.